



// CASE STUDY & TECHNICAL DOCUMENTATION

Know the instant something goes down.

A full-stack uptime & latency monitor for early-stage founders —
built as part of the **52 Apps Challenge**: web app, native iOS
companion, and everything it took to actually ship it.

WEB

Node.js · Express ·
MongoDB |

iOS

SwiftUI · Swift
Charts |

INFRA

Railway ·
Resend

What is PulsePing?

PulsePing is a system health monitor built for founders who need certainty, not guesswork, that their product is live. It checks endpoints on a real schedule, keeps a full history of every response, and surfaces status changes the moment they happen — on the web, and natively on iOS.



Who it's for

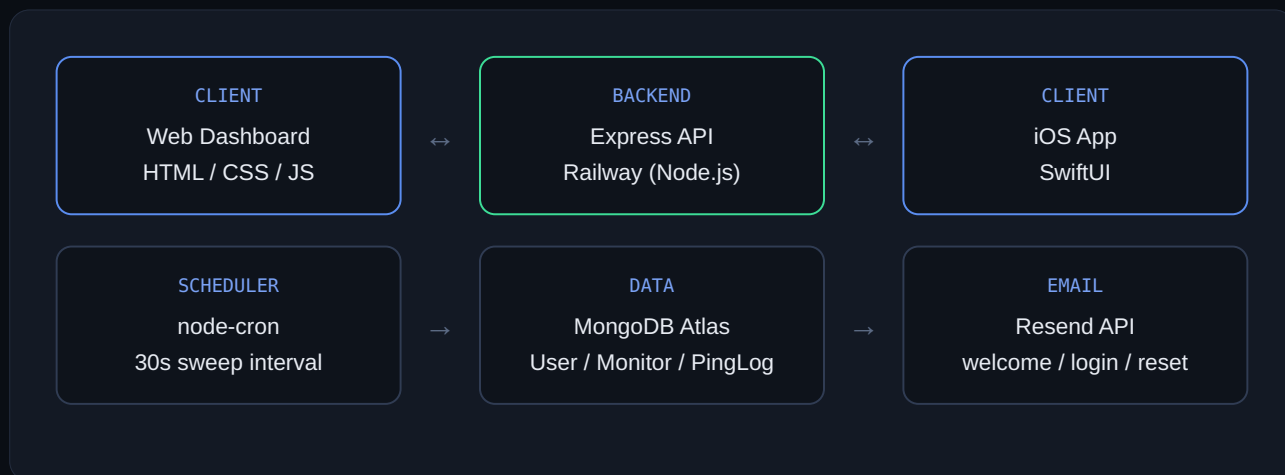
An early-stage founder running a small API or web app who wants a single, trustworthy source of truth for uptime — without paying for or configuring an enterprise observability stack. Add a URL, get a live status page you can share, and get pinged the moment something breaks.

The core loop

Add a monitor → a background scheduler checks it on its own interval → every check is logged with latency, status code, and timestamp → the dashboard and public status page render that history as uptime % and a live latency chart → if status flips, that's the trigger point for a push notification (phase 2, pending a paid Apple Developer account).

System overview

One backend, two clients. The web app and iOS app hit the same REST API and the same MongoDB instance — there's no divergence in business logic between platforms, only in how each renders it.



Data model

COLLECTION	PURPOSE	KEY FIELDS
User	Auth & account	email, hashed password, resetPasswordToken, deviceTokens[]
Monitor	A tracked endpoint	url, intervalSeconds, publicSlug, status, totalChecks / totalUpChecks (for O(1) uptime%)
PingLog	One row per health check	status, latencyMs, statusCode, checkedAt (compound-indexed on monitor + time)

Why a cron sweep instead of per-monitor timers

A single scheduler tick every 30 seconds scans all active monitors and checks whichever are "due" based on their own `intervalSeconds`. This avoids holding hundreds of individual `setInterval` timers in memory as the monitor count grows, and survives a server restart cleanly — nothing to re-register.

Public status pages

Every monitor gets a unique, unguessable `publicSlug` (via nanoid) and a shareable URL — `/s/:slug` — that serves status, uptime %, and latency history with zero authentication. This is also the endpoint the iOS Home Screen Widget is designed to poll, since a widget shouldn't need to hold a JWT.

What actually shipped

Web application

- **Auth** — register, login, forgot/reset password (email token, 1hr expiry, hashed in DB), change password, rate-limited against brute force
- **Monitor management** — add, edit interval, delete, manual "check now," per-monitor public/private toggle
- **Live dashboard** — polls every 15s, animated pulse-line status indicator, latency + uptime at a glance
- **Latency history** — Chart.js graph of the last 50 checks per monitor
- **Public status pages** — no-login shareable link per monitor, auto-refreshing
- **Transactional email** — welcome on signup, security alert on every login (time + IP), password reset

iOS application (SwiftUI)

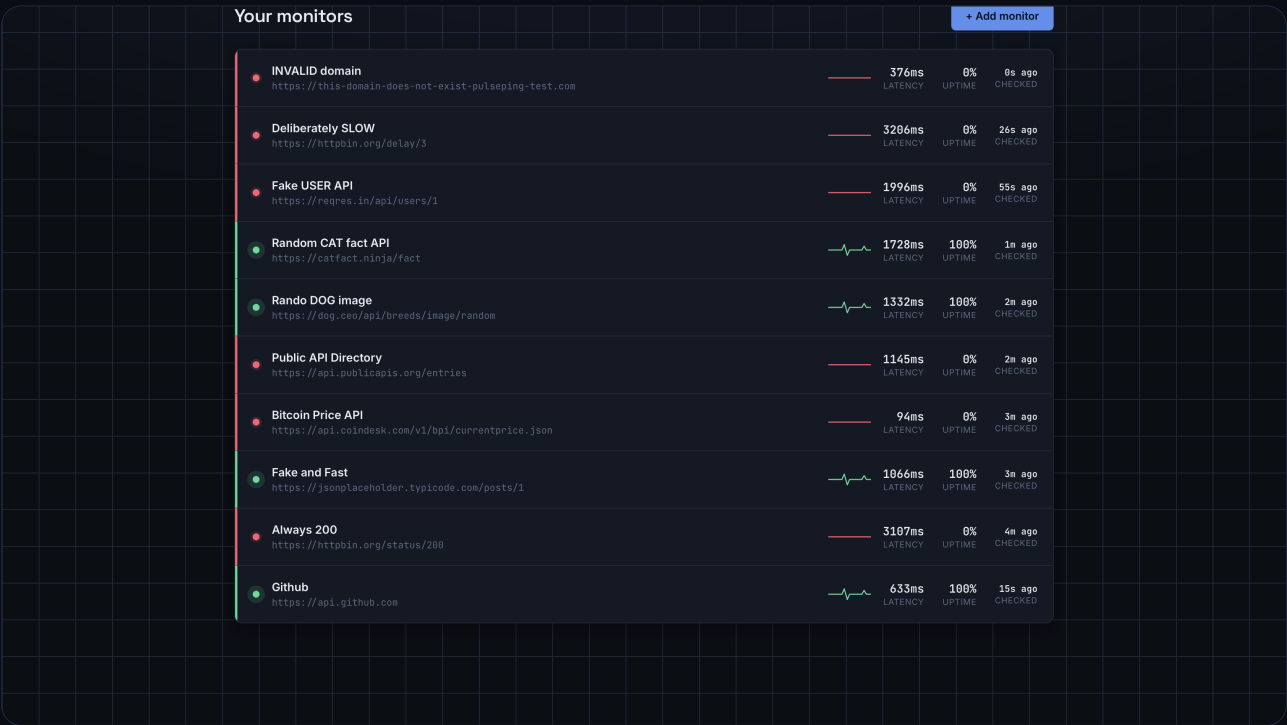
- **Full auth flow** — native login/register, Keychain-backed JWT storage, forgot password (hands off to the web reset flow via browser)
- **Monitor list & detail** — same data as web, native Swift Charts latency graph, swipe-to-delete
- **Settings screen** — editable backend URL (dev/prod switching without a rebuild), change password, logout
- **Design parity** — colors, type scale, and the pulse-line motif ported 1:1 from the web CSS tokens into a Swift theme file
- **Scaffolded, not yet live** — Home Screen Widget and push notifications are fully coded but gated behind Apple's paid Developer Program (App Groups and Push Notifications entitlements aren't available on a free/personal team)

Backend / infrastructure

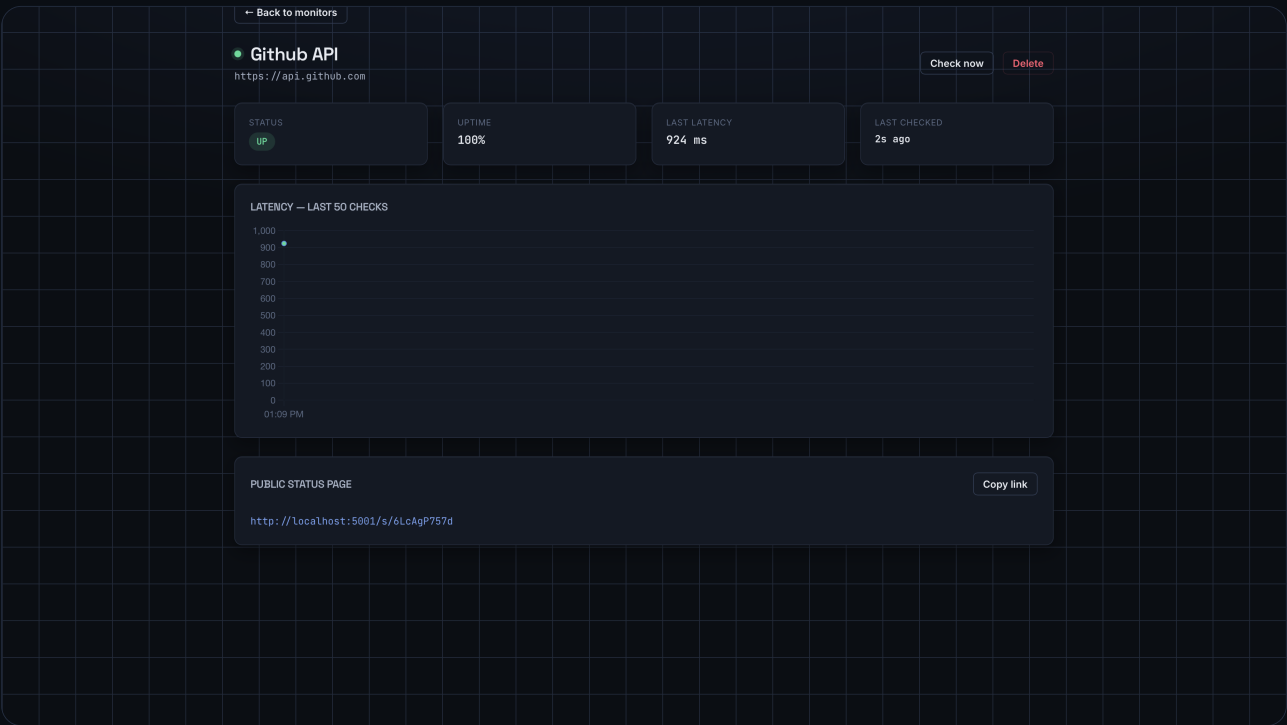
- **REST API** — 9 endpoints across auth, monitors, and public status
- **Background ping engine** — real HTTP checks with latency timing, status classification, and automatic status-flip detection
- **Security** — bcrypt password hashing, JWT auth, rate limiting on auth routes, hashed (not plaintext) password-reset tokens
- **Deployed on Railway** — connected to GitHub for auto-deploy on push, environment-variable-driven config

Web dashboard

Dark "mission control" theme — deliberately steering away from the generic near-black-plus-neon-green AI-dashboard look with a distinct pulse/heartbeat motif tied to the product name.

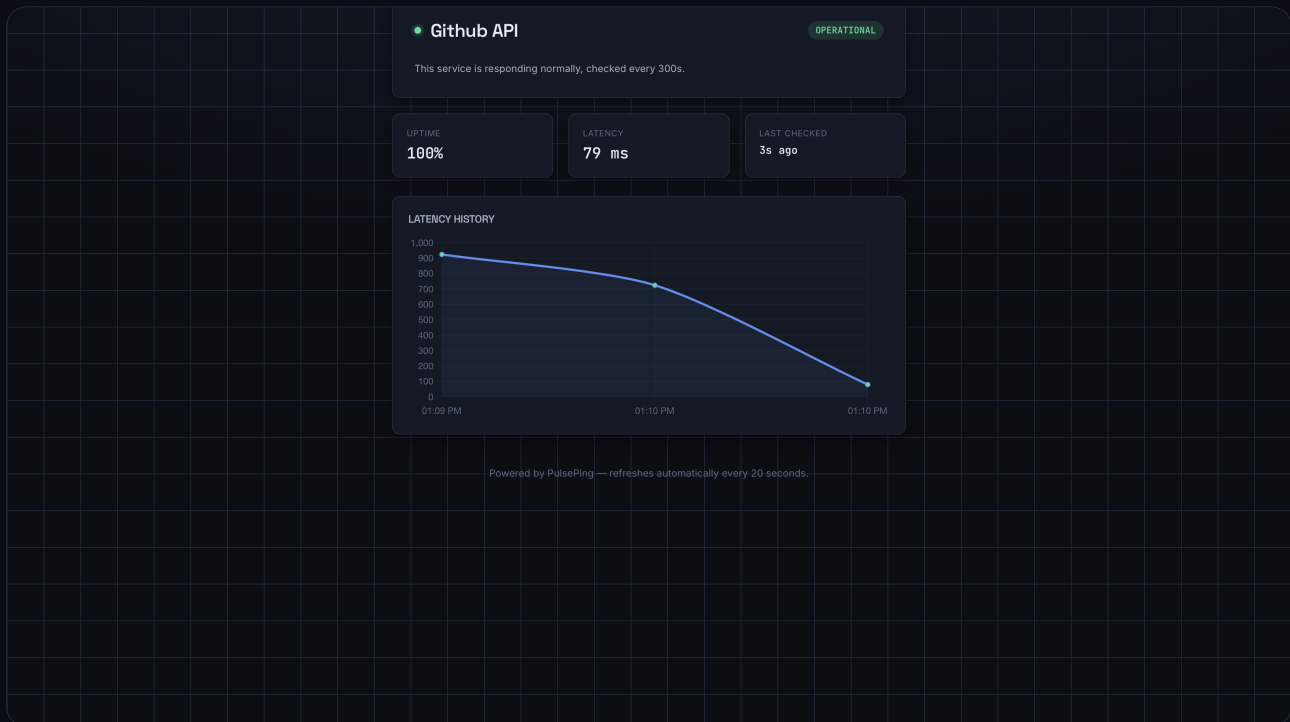


Live monitor list — animated pulse-line reflects real up/down status

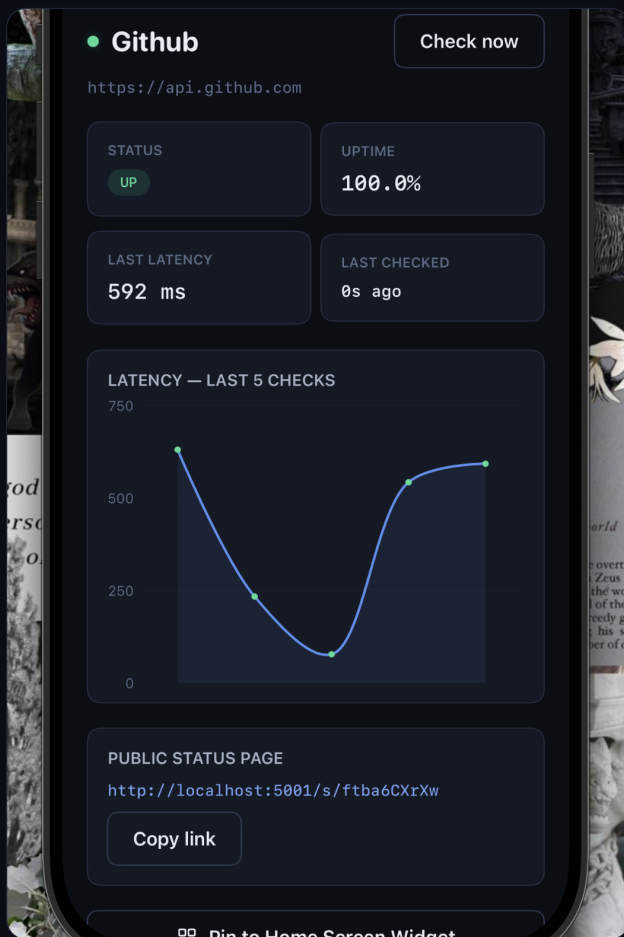


Monitor detail — latency history chart, uptime %, public status link

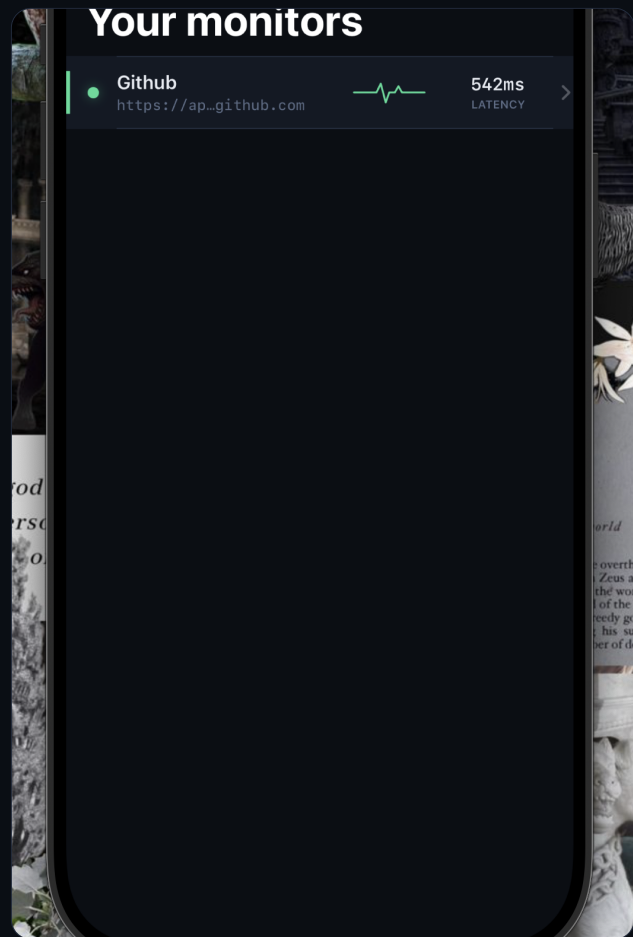
Public status page & iOS app



Public status page – no login required, shareable link per monitor



iOS — monitor list



iOS — Swift Charts latency detail

Five real problems, fixed live

The code compiling was never the hard part. Getting it to actually run correctly in someone else's environment — Xcode, Railway, Apple's ecosystem — was. These are the ones worth remembering.

01 A committed `.env` file, and the lesson about deleting vs. rotating

A missing local `.gitignore` let a `.env` file — MongoDB password, JWT secret, email app password — get pushed to a public GitHub repo. Deleting the file in a follow-up commit doesn't remove it from git history; anyone could still view it in an old commit.

Fix: rotated every exposed credential (new DB password, new JWT secret, new email app password), then nuked the local `.git` folder entirely and re-initialized a clean repo rather than trying to scrub history.

02 Railway couldn't find the app to build

Railway's builder (Railpack) scanned the repo root and found only a `.gitignore` — the actual `package.json` was one level down in a `PulsePing/` subfolder, a byproduct of how the repo was initialized.

Fix: set the service's **Root Directory** in Railway's settings, pointing the build at the correct subfolder.

03 The freshly deployed domain wouldn't resolve — anywhere

After generating a Railway domain, the site returned "server not found" in every browser and even `curl`. A manual DNS query against Google's resolver (8.8.8.8) returned a valid A record immediately — the domain was live. The local machine's own DNS (routed through a phone hotspot) simply wasn't resolving it.

Fix: pointed the Mac's Wi-Fi DNS servers at 8.8.8.8 / 8.8.4.4 directly in Network settings — confirmed the issue was local, not the deployment.

04 Emails failed in production only, with an IPv6 red herring

Nodemailer + Gmail SMTP worked flawlessly locally but failed on Railway with `ENETUNREACH` against an IPv6 address. Forcing IPv4 in the transport config, then forcing Node's global DNS resolution order, both still resulted in failure — just a plain `connection timeout` instead. That was the real signal: Railway's network blocks outbound SMTP ports entirely, a common anti-spam measure on cloud platforms — no client-side IP setting could fix it.

Fix: replaced raw SMTP with **Resend's HTTPS API** for transactional email — HTTPS (443) is never blocked the way SMTP ports are.

Apple Silently blocked the iOS Widget and Push setup

Adding an App Group capability in Xcode (required for the Home Screen Widget to share data with the main app) failed with "Communication with Apple failed / No profiles found." The actual cause: App Groups and Push Notifications entitlements are restricted to a paid Apple Developer Program membership — a free "Personal Team" account cannot register them, regardless of configuration.

Fix: shipped the core app without those two features, moved the fully-written widget code into a clearly labeled `Phase2-Optional` folder with its own setup notes, and deferred until enrolling in the paid program.

What this build reinforced

Credential hygiene is a day-zero task, not a cleanup task

`.gitignore` has to exist and be correct before the first commit, not after. And once a secret is exposed — even briefly, even in a repo nobody's watching — the only safe move is rotation, not deletion.

"It works locally" and "it works in production" are different claims

Three of the five real bugs here (Railway root dir, DNS, SMTP blocking) only existed because the deployment environment's network and filesystem assumptions differed from a laptop's. None of them would show up in local testing, however thorough.

Free tiers have real, undocumented-until-you-hit-them walls

Cloud SMTP blocking and Apple's free-developer-account entitlement limits both look like bugs until you research them — they're policy, not accidents. Budgeting for that (an HTTPS email API instead of SMTP, a paid Apple account before promising push notifications) is part of scoping a real product, not just writing code for one.

What's next

- **Verify a custom domain on Resend** — currently email can only be sent to the developer's own verified address; a verified sending domain removes that limit for real users
- **iOS Home Screen Widget** — code is complete, gated on a paid Apple Developer Program enrollment
- **Push notifications** — backend APNs service is built and wired to status-flip detection; needs the same paid account plus a device to test on (Simulator can't receive real pushes)
- **Per-widget configuration** — upgrade from one shared "pinned monitor" to per-instance widget configuration via `WidgetConfigurationIntent`

Live app: pulseping-production.up.railway.app · Source: github.com/Guywhocanbuild/pulseping · Built as part of the 52 Apps Challenge